

GUÍA DE DESARROLLO E IMPLEMENTACIÓN

Evaluación Parcial N.º 1 (EP1)

CASO 0 — PEDIDOS360

React 19 + Vite + TypeScript + React Router + MSAL + Amazon Cognito

AWS API Gateway + BFF Spring Boot + Catalog + Orders + Oracle

Campo	Detalle
Docente	Patricio Silva Gutiérrez
Proyecto	Pedidos360
Frontend real	React 19 + Vite 8 + TypeScript + React Router 7
Identidad	Microsoft Entra ID y Amazon Cognito coexistiendo
Backend	BFF, Catalog y Orders en Spring Boot 3.5 / Java 17
Persistencia	Oracle para Catalog y Orders

Adaptación técnica: esta versión describe el frontend React entregado por el grupo. No utiliza Angular, ``AppModule``, ``MsalInterceptor`` ni componentes Angular. El caso 0 es el único caso implementado por este equipo; los casos 1 a 6 de la guía original son alternativas de otros dominios.

Docente: Patricio Silva Gutiérrez

1. Contexto y objetivo de la EP1

La EP1 demuestra una integración segura y funcional entre identidad, frontend React, API Gateway, BFF y los microservicios mínimos del dominio. La solución debe conservar los módulos necesarios para diferenciar a los actores y demostrar autenticación, autorización y reglas de negocio reales.

Objetivos del caso 0

- Permitir el acceso con Microsoft Entra ID para las rutas **/api/****.
- Permitir el acceso con Amazon Cognito para las rutas **/aws/api/****.
- Proteger las rutas privadas de React y ocultar acciones no permitidas según el rol.
- Validar el JWT en API Gateway, BFF y microservicios.
- Aplicar autorización por `Admin`, `Operador` y `Cliente`.
- Demostrar pedidos, catálogo, stock y transiciones sobre Oracle.
- Entregar el código en repositorios separados y con GitHub Actions.

Stack utilizado

Capa	Tecnología
Frontend	React 19, Vite 8, TypeScript 5.9, React Router 7
Identidad	MSAL para Entra; Amplify/OIDC para Cognito
Gateway	AWS API Gateway HTTP API con dos JWT authorizers
Backend	Spring Boot 3.5, Spring Security, Java 17
Datos	Oracle; Docker Oracle solo para desarrollo local
Entrega	GitHub Actions, GitHub OIDC, Amazon ECR y Terraform

2. Arquitectura obligatoria

```
Navegador
+-- Sesion Entra -----> /api/**
+-- Sesion Cognito ----> /aws/api/**
    | Authorization: Bearer <access_token>
    v
API Gateway HTTP API
+-- authorizer Entra
+-- authorizer Cognito
    | red privada
    v
BFF Spring Boot (sin JDBC)
+-- Catalog -- Oracle
+-- Orders -- Oracle
```

Regla arquitectónica

El BFF no se conecta directamente a la base de datos. Valida el token, aplica autorización y coordina las llamadas a Catalog y Orders. API Gateway es la única URL pública del backend; el frontend es una superficie pública distinta.

Separación de namespaces

Namespace	Proveedor	Callback SPA	Validador
/api/**	Microsoft Entra ID	/login	Issuer/audience Entra; claim roles

Namespace	Proveedor	Callback SPA	Validador
/aws/api/**	Amazon Cognito	/auth/cognito/callback	Issuer Cognito; audience/client_id; token_use=access; cognito:groups

Un token de Entra no se acepta en **/aws/api/**** y un token de Cognito no se acepta en **/api/****. El frontend mantiene contextos separados y no envía el token de un proveedor al namespace del otro.

3. Estructura real de repositorios

El proyecto se entrega en cinco repositorios públicos separados. Esta es la estructura que debe reflejarse en la evidencia y en la guía:

Repositorio	Responsabilidad
pedidos360-frontend	SPA React, MSAL, Cognito, rutas, guards, páginas y cliente HTTP.
pedidos360-bff	Validación JWT dual, autorización, CORS y proxy privado.
pedidos360-catalog	Productos, stock y reservas idempotentes.
pedidos360-orders	Pedidos, estados, propiedad del cliente y coordinación de stock.
pedidos360-infra	Docker Compose, OpenAPI, Terraform, documentación y GitHub Actions de AWS.

Estructura local de carpetas

```
Pedidos360-repos/
+-- pedidos360-frontend/
+-- pedidos360-bff/
+-- pedidos360-catalog/
+-- pedidos360-orders/
+-- pedidos360-infra/
```

Comandos de verificación

```
npm --prefix ../pedidos360-frontend ci
npm --prefix ../pedidos360-frontend test
npm --prefix ../pedidos360-frontend run build
mvn -f ../pedidos360-bff/pom.xml clean verify
mvn -f ../pedidos360-catalog/pom.xml clean verify
mvn -f ../pedidos360-orders/pom.xml clean verify
```

No se versionan `.env``, tokens, contraseñas, ``.tfvars`` ni estados de Terraform.

4. Caso 0: actores y módulos

Actores

Actor	Responsabilidad
Admin	Administra productos, stock, pedidos y la operación global.
Operador	Gestiona pedidos, cambios de estado y operación diaria.
Cliente	Crea pedidos y consulta/cancela sus propios pedidos.

Módulos mínimos

Módulo	Contenido	Microservicio
Gestión de pedidos	CRUD y estados CREADO, ACEPTADO, EN_PREPARACIÓN, DESPACHADO, ENTREGADO, CANCELADO.	pedidos360-orders
Catálogo	CRUD de productos, precios y stock.	pedidos360-catalog

Reglas de negocio obligatorias

- No se puede **DESPACHAR** un pedido que no esté en **EN_PREPARACIÓN**.
- Al aceptar un pedido se reserva o descuenta stock.
- Al cancelar un pedido con reserva se restituye el stock.
- Las reservas son idempotentes por `orderId`.
- Un Cliente solo puede consultar y cancelar sus propios pedidos mientras estén en `CREADO`.
- Un Cliente puede ver productos para crear un pedido, pero no puede administrar catálogo ni stock.

Matriz de permisos

Acción	Admin	Operador	Cliente
Crear pedido	Sí	Sí	Sí
Ver sus propios pedidos	Sí	Sí	Sí
Ver todos los pedidos	Sí	Sí	No
Cambiar estado	Sí	Sí	No
Crear/editar producto	Sí	Sí	No
Administrar stock	Sí	Sí	No

La UI oculta controles no permitidos, pero la autorización real se aplica en BFF, Catalog y Orders.

5. Frontend React entregado

Componentes y guards

Archivo	Responsabilidad
src/main.tsx	MsalProvider, AuthProvider, CognitoAuthProvider y BrowserRouter.
src/App.tsx	Rutas /login, /dashboard, /orders, /catalog, /aws y callback Cognito.
src/auth/msal.ts	Configuración MSAL para Entra ID.
src/auth/AuthContext.tsx	Sesión Entra, claims, roles, oid, login y logout.
src/auth/CognitoAuthContext.tsx	Sesión Cognito, grupos, login, logout y access token.
src/auth/cognito.ts	Authorization Code + PKCE S256, sin client secret, sessionStorage.
src/auth/RedirectHandler.tsx	Callback de Entra en /login.
src/auth/RequireAuth.tsx	Guard de rutas Entra.
src/auth/RequireCatalogRole.tsx	Guard de catálogo para Admin/Operador.
src/auth/RequireCognitoAuth.tsx	Guard independiente del portal /aws.
src/pages/CognitoCallbackPage.tsx	Procesa el retorno OAuth de Cognito.
src/pages/AwsPortalPage.tsx	Portal Cognito que usa /aws/api/**.
src/api/client.ts	fetch, Authorization Bearer y errores HTTP.
src/api/services.ts	Contratos de pedidos y catálogo.

Rutas React

Ruta	Acceso	Componente
/login	Público	LoginPage: Microsoft o Amazon Cognito
/auth/cognito/callback	Público	CognitoCallbackPage
/dashboard	Entra autenticado	DashboardPage
/orders	Entra autenticado	OrdersPage
/catalog	Admin u Operador	CatalogPage
/aws	Sesión Cognito	AwsPortalPage

Cliente HTTP y tokens

El cliente React agrega `Authorization: Bearer <access\_token>` a las rutas del proveedor activo. Las rutas normales usan `/api/\*\*` y el portal Cognito usa exclusivamente `/aws/api/\*\*`. No se envía el ID token de Cognito a la API.

Pruebas frontend

La suite React cubre claims, roles estrictos, cliente HTTP, servicios, login Entra, login Cognito, callback, guard Cognito y las rutas del portal AWS.

6. Backend y seguridad

BFF

- Dos cadenas Spring Security separadas por prefijo.

- Valida firma, issuer, audience, expiración y `nbf` de Entra.
- Valida issuer Cognito, JWKS, audience/client ID y `token\_use=access`.
- Convierte `roles` y `cognito:groups` en `ROLE\_Admin`, `ROLE\_Operador` y `ROLE\_Cliente`.
- Reenvía a Catalog y Orders únicamente dentro de la red privada.
- Conserva el prefijo `/api` o `/aws/api` al reenviar.
- No tiene JDBC, datasource ni credenciales de Oracle.

Catalog

- Expone productos bajo `/api/catalog/\*\*` y `/aws/api/catalog/\*\*`.
- CRUD y stock solo para Admin u Operador.
- Las reservas internas no se publican en API Gateway.
- Valida los roles del claim correcto para el namespace recibido.

Orders

- Expone pedidos bajo `/api/orders/\*\*` y `/aws/api/orders/\*\*`.
- Aplica propiedad del pedido para Cliente.
- Impide transiciones operacionales a Cliente.
- Usa `oid` para Entra y `sub` para Cognito.
- Propaga el mismo Bearer validado a Catalog en el namespace correcto.

Rutas públicas del caso 0

Operación	Entra	Cognito
Pedidos	/api/orders/**	/aws/api/orders/**
Productos	/api/catalog/products/**	/aws/api/catalog/products/**
Reservas internas	/internal/catalog/stock/reservations/**	/aws/api/internal/catalog/stock/reservations/**

7. API Gateway, AWS y entrega

API Gateway

- Authorizer JWT de Entra para `/api/\*\*`.
- Authorizer JWT de Cognito para `/aws/api/\*\*`, con issuer y App Client público.
- Rutas Cognito con scope `openid` para rechazar ID tokens sin `scope`.
- CORS con orígenes exactos y preflight OPTIONS.
- No existe ruta `\$default` ni se exponen microservicios internos.

Terraform y ECR

Terraform crea la HTTP API, los authorizers, las rutas, el VPC Link y repositorios ECR cifrados con scan-on-push. La red privada, ECS/Fargate, Oracle, Secrets Manager, IAM y la integración privada del BFF deben completarse antes de aplicar en producción. No se deben inventar credenciales ni guardar secretos en GitHub.

Valores públicos de Cognito

```
COGNITO_USER_POOL_ID=us-east-1_UmEhPRYdI
COGNITO_USER_POOL_CLIENT_ID=59be26pgg5ginu2sutr8eetgjj
COGNITO_ISSUER=https://cognito-idp.us-east-1.amazonaws.com/us-east-1_UmEhPRYdI
COGNITO_REDIRECT_URI=http://localhost:4200/auth/cognito/callback
COGNITO_LOGOUT_URI=http://localhost:4200/login
```

GitHub Actions y OIDC

Los workflows de aplicación construyen imágenes y las publican en ECR usando GitHub OIDC. El workflow de Terraform genera un plan y solo aplica después de la protección del environment. No se almacenan AWS access keys.

8. Pruebas y evidencias obligatorias

Resultado automatizado del equipo

Componente	Pruebas	Resultado
pedidos360-frontend	23	Correctas
pedidos360-bff	21	Correctas
pedidos360-catalog	19	Correctas
pedidos360-orders	33	Correctas
Total	96	0 fallos

Matriz mínima de seguridad

Caso	Resultado esperado
Sin token	401
Token expirado o audience incorrecta	401
Token Entra válido en /aws/api	401
Token Cognito válido en /api	401
Cognito ID token	401
Cliente intenta escritura/admin	403
Transición que salta ACEPTADO	409

Caso	Resultado esperado
Cantidad de pedido inválida	400
Aceptar pedido con stock suficiente	200 y stock actualizado

Evidencias que se deben guardar

- Login Entra y login Cognito funcionando, sin mostrar tokens completos.
- Callbacks y claim `cognito:groups` verificados de forma saneada.
- Respuestas 401, 403, 409 y 200 por actor.
- Pruebas de stock antes/después de aceptar y cancelar.
- Plan de Terraform revisado y outputs sanitizados.
- Digest de imágenes ECR y estado de los servicios privados.
- Evidencia de que BFF, Catalog, Orders y Oracle no tienen ingress público.

9. Ejecución local del caso 0

Requisitos

- Docker Engine y Docker Compose v2.
- JDK 17 y Maven 3.9 o superior.
- Node.js 26 y npm.
- Microsoft Entra ID configurado para la SPA y la API.
- Cognito App Client público configurado con PKCE S256.

Arranque

```
cd Pedidos360-repos/pedidos360-infra
cp .env.example .env
# completar valores locales sin versionar el archivo
./scripts/run-local.sh
```

La URL local del frontend es `http://localhost:4200`. El BFF escucha en `http://localhost:8080`. Catalog, Orders y Oracle permanecen en redes privadas. El script de smoke verifica el flujo de pedidos, stock y transiciones.

Probar Cognito localmente

- Registrar `http://localhost:4200/auth/cognito/callback` como callback exacto.
- Registrar `http://localhost:4200/login` como sign-out URL.
- Abrir `/login` y seleccionar **Acceder con Amazon Cognito**.
- Verificar que el retorno termine en `/aws` y no en `/login`.
- Si aparece `redirect\_mismatch`, corregir la lista de URLs del App Client.

10. Checklist final del caso 0

- [] La entrega corresponde al Caso 0: Pedidos360.
- [] El frontend usa React, no Angular.
- [] Existen cinco repositorios separados.
- [] El frontend ofrece login Entra y login Cognito.
- [] Los callbacks de Entra y Cognito están separados.
- [] El frontend no usa client secrets.
- [] Las rutas `/api/\*\*` y `/aws/api/\*\*` están protegidas por separado.
- [] Los roles Admin, Operador y Cliente producen diferencias reales.
- [] El BFF no accede directamente a Oracle.
- [] Las transiciones de pedidos y las reglas de stock funcionan.
- [] API Gateway es la única URL pública del backend.
- [] Las imágenes se publican en ECR mediante GitHub OIDC.
- [] Terraform fue validado y revisado antes de cualquier apply.
- [] Las evidencias no contienen tokens completos ni secretos.

Resumen: Pedidos360 demuestra un frontend React con dos proveedores de identidad, autorización por roles, API Gateway, BFF, microservicios Spring Boot, Oracle y entrega automatizada. El caso 0 se entrega con los módulos de

pedidos y catálogo, no con los casos alternativos de la guía original.